

MODE D'EMPLOI INSTALLATION

D'UN SERVEUR SOUS GNU/LINUX

Ce document traitera de la mise en place d'un serveur fonctionnel dans un réseau. Veuillez porter une particulière attention aux étapes de cette installation.

Rédacteurs :

BOUSSAID Ilian,

BAVYE Bastien

Informations importantes : Dans chaque commande, vous verrez « sudo », cela permet d'exécuter une commande en administrateur et ainsi éviter d'avoir une erreur de permissions. A votre première exécution (après démarrage) avec cette commande, il vous sera demandé le mot de passe « sudo » : il s'agit de votre mot de passe de session. Tapez-le simplement et faites [Entrée] (le mot de passe que vous entrez paraît invisible, mais il est bien pris en compte). Faites aussi attention au répertoire d'où sont exécutées les commandes. Ne prenez pas en compte les sessions des captures d'écran : elles ont été faites sur plusieurs postes différents, mais le résultat ne change pas. Si vous voulez approfondir les commandes utilisées, voir annexe.

Table des matières

1. Installation et configuration d'un serveur local.....	3
1.1 Prérequis	3
1.2 Installation et lancement d'apache2 dans le terminal GNU/Linux	3
1.3 Placement de sites internet en local	4
1.3.1 Vérification d'apache2 sur l'ordinateur	4
1.3.2 Installation des sites dans apache2	5
1.3.3 Lancement du site internet en local	8
1.4 Lancement de WireShark	9
1.4.1 Définition des trames	9
1.4.2 Accéder aux trames internet	9
1.5 Passage au port 8080	10
1.6 Capture de formulaire GET/POST	12
1.7 Configuration avec Telnet	12
2. Configuration d'un second site avec un DNS et des Hosts	14
3. Configuration d'un serveur local Windows (WAMP)	16
3.1 Installation de WampServer	16
3.2 Dépôt des fichiers dans le serveur local	16
3.3 Lancement et test du serveur Windows	19
4. Mise en place d'un serveur avec Nginx	20
4.1 Installation de Nginx	20
4.2 Configuration de sites pour Nginx	20
4.3 Lancement et vérification du site en local	21
5. Mise en place d'un site sécurisé avec SSL	23
Annexe	25

1. Installation et configuration d'un serveur local

1.1 Prérequis

Pour pouvoir réaliser toutes les procédures suivantes, il va vous falloir quelques **prérequis**.

Matériels nécessaires :

- Un **ordinateur**,
- Un **système d'exploitation Linux** et/ou **Windows préinstallé** ou une machine virtuelle avec l'iso Ubuntu (VMWare est un logiciel qui peut être gratuit, vous pouvez opter pour la version professionnelle),
- Avoir la **dernière version logicielle d'Ubuntu ou Windows** (il vous sera proposé de mettre celui-ci à jour dès votre première connexion à votre machine),
- Une **connexion filaire, ou non-filaire**.

Une fois tout installé, vous pouvez commencer les manipulations.

1.2 Installation et lancement d'apache2 dans le terminal GNU/Linux

Afin d'avoir la possibilité de pouvoir lancer un serveur, il faut tout d'abord installer **Apache2**, directement intégré dans le « **apt** » de GNU/Linux. Le « **apt** » est la bibliothèque intégrée dans la machine, mais il faut tout de même installer le logiciel. Pour cette étape, une connexion internet est **INDISPENSABLE**.

Pour ce faire, allez sur votre terminal, en appuyant simultanément sur [Ctrl] + [Alt] + T. Le terminal s'ouvre. Ensuite, tapez cette commande :

```
devops@ubuntu:~$ sudo apt install apache2
```

Faites [Entrée]. C'est alors que des lignes de commandes s'exécutent. N'y prêtez pas **attention**, apache2 s'installe sur votre ordinateur. De ce fait, n'éteignez pas votre ordinateur et ne touchez pas à votre clavier mis à part pour confirmer l'installation (le terminal vous demandera de taper **Y/N** (soit Y pour Yes, soit N pour No), appuyez sur Y puis [Entrée]).

Dès que le téléchargement est complété, vous pourrez de nouveau écrire dans votre terminal. Il va falloir **lancer apache2** pour ainsi pouvoir accéder à un serveur local.

Restez dans le terminal et exécuter cette commande :

```
devops@ubuntu:~$ sudo systemctl start apache2
```

C'est alors qu'apache2 est lancé et que votre site peut enfin être mis sur le serveur.

N.B. : Dans le cas où vous êtes sur une machine virtuelle et que la commande produit une erreur, alors il faut arrêter le processus « nginx » lancé dès le démarrage. Pour l'arrêter, faites cette commande dans le terminal :

```
devops@ubuntu:~$ sudo systemctl stop nginx  
[sudo] password for devops:
```

Ainsi, vous pourrez réexécuter la commande ci-dessus.

Si vous voulez arrêter à tout moment le processus apache2, il faut exécuter la commande :

```
devops@ubuntu:~$ sudo systemctl stop apache2
```

N.B. : Plus aucunes pages de vos sites ne seront **accessibles** (cf. **Placement de sites internet dans le réseau**).

1.3 Placement de sites internet en local

1.3.1 Vérification d'apache2 sur l'ordinateur

Il est maintenant temps de **placer** vos sites internet dans le réseau. Cette partie sera détaillée au mieux pour comprendre les manipulations et pouvoir les réitérer.

Tout d'abord, commencez par voir si apache2 est bien **présent** dans l'ordinateur. La commande « cd » permet de se déplacer et « ls » de voir le contenu d'un répertoire. Sur votre terminal, exécutez ces commandes :

Déplacement vers le répertoire d'apache2 et affichage :

```
devops@ubuntu:~$ cd /etc/apache2  
devops@ubuntu:/etc/apache2$ ls  
apache2.conf  conf-enabled  magic          mods-enabled  sites-available  
conf-available  envvars      mods-available  ports.conf    sites-enabled
```

Si vous arrivez à y accéder, alors apache2 est **bien installé**. Vous pouvez voir des textes écrits en bleu et d'autres en blanc. Les textes bleus représentent les **dossiers** (accessibles via la commande « cd » [nom_repertoire] (en fonction du répertoire dans lequel vous êtes)), et les textes en blanc représentent les **fichiers** (impossible de faire la commande « cd »).

Ensuite, le dossier qui nous intéresse se nomme : **sites-available** : il permettra de configurer un fichier qui récupère le dossier parent des pages internet. Pour y accéder, tapez les commandes :

```
devops@ubuntu:/etc/apache2$ cd sites-available  
devops@ubuntu:/etc/apache2/sites-available$ ls  
000-default.conf  default-ssl.conf
```

Deux fichiers sont déjà présents : un pour héberger des sites http (000-default.conf non sécurisé), et un autre pour des sites https (default-ssl.conf sécurisé). Celui qui nous intéresse est **000-default.conf**.

N.B. : Pour pouvoir héberger plusieurs sites différents, qui n'ont aucun lien entre elles, vous pouvez créer un nouveau fichier conf et le modifier avec la commande nano [nom_fichier]. Ici, il s'appellera, en guise d'exemple site1 :

```
devops@ubuntu:/etc/apache2/sites-available$ sudo touch site1.conf
devops@ubuntu:/etc/apache2/sites-available$ ls
000-default.conf  default-ssl.conf  site1.conf
```

Pour modifier :

```
devops@ubuntu:/etc/apache2/sites-available$ sudo nano site1.conf
```

1.3.2 Installation des sites dans apache2

Nous avons vu précédemment que sites-available contenait 000-default.conf. En réalité, ce fichier permet d'identifier le répertoire où les fichiers (pages internet) qui nous intéressent et donc de créer un lien. Pour modifier le contenu du fichier, vous devez faire cette commande :

```
devops@ubuntu:/etc/apache2/sites-available$ sudo nano 000-default.conf
```

Après avoir fait entrée, beaucoup d'informations, dont des commentaires, sont présentes. N'y prêtez pas plus d'intentions que cela car seulement 1 ligne précise vous intéresse : le **DocumentRoot**. Veuillez rajouter, ou non, votre choix, un dossier en plus après le « /html ». Il est important à noter que si vous faites le choix de mettre les fichiers dans un autre répertoire, il faut **impérativement** se souvenir du nom de répertoire que vous avez indiqué. Le DocumentRoot est très important pour la suite.

Exemple :

```
ServerAdmin webmaster@localhost
DocumentRoot /var/www/html/site1/
```

Ici, les sites que nous voudrions héberger seront dans le répertoire site1 (**ATTENTION** : aucun lien entre site1.conf et celui-ci !). Nous allons donc retenir ce nom pour les prochaines manipulations.

Avec ceci de fait, il faut alors créer un répertoire site1 à un endroit **particulier**. En effet, comme vous pouvez le voir sur l'illustration précédente, le DocumentRoot montre qu'il va chercher dans le répertoire « /var/www/html » (site1 est ignoré car il n'est pas encore créé). Cela signifie qu'il

faut **créer un répertoire** à partir de ce chemin. De plus, la commande `mkdir [chemin]/nom_repertoire` permet de créer un répertoire à partir d'un chemin donné, et où le « nom_repertoire » représente le nom de notre répertoire. Pour simplifier les prochaines manipulations, **nous resterons dans le répertoire actuelle** (le répertoire n'est pas à changer, donc aucun « cd »).

C'est alors que la commande à effectuer est :

```
devops@ubuntu:/etc/apache2/sites-available$ sudo mkdir /var/www/html/site1
```

N.B. : Pour l'exemple, le site nommé est site1 car ce nom est le même que le nom spécifié pour le répertoire dans le DocumentRoot. Si vous aviez spécifié un autre nom, comme « loli » dans le DocumentRoot, alors vous devriez faire :

mkdir /var/www/html/loli

En règle générale, le répertoire suivant « html/ » est celui que vous aviez **indiqué** dans le DocumentRoot.

Désormais, les liens ont été effectués, il manque plus qu'à placer les sites internet dans le répertoire pour qu'il s'affiche en local. Les sites sont supposés déjà créés et qu'il ne faut plus que les copier. La commande pour copier a pour nom : **cp** (comme copy).

La commande générale est :

cp [element_source] [element_destination]

Exemple :

```
devops@ubuntu:/etc/apache2/sites-available$ sudo cp /media/devops/BASTIEN/jeu/index.html /var/www/html/site1/index.html
```

Ici, l'élément source est `/media/devops/BASTIEN/jeu/index.html`

(Le terminal cherche dans la clé USB (media), dans l'utilisateur actuel (ici, devops), le fichier (index.html) dans le chemin (BASTIEN/jeu/).

L'élément destination est `/var/www/html/[nom_repertoire]/index.html`

(Il faut **OBLIGATOIREMENT** placer dans `/var/www/html/` avec [nom_repertoire], le nom du répertoire donné pour le DocumentRoot après le html/. Si vous voulez copier le fichier avec le même nom, alors le nom du fichier après nom_repertoire n'est pas obligatoire, sinon, il l'est. Il est **OBLIGATOIRE** de nommer le fichier principal du site « index.html » : c'est le nom que le fichier **.conf** va chercher).

En guise d'exemple, il est possible d'ajouter le css d'une page en faisant exactement la même manipulation :

```
devops@ubuntu:/etc/apache2/sites-available$ sudo cp /media/devops/BASTIEN/jeu/style_index.css /var/www/html/site1/style_index.css
```

Votre **site** et son **style** est maintenant disponible en local.

Grâce à « ls », il est possible de vérifier dans site1 si les deux fichiers sont présents.

```
devops@ubuntu:/etc/apache2/sites-available$ ls /var/www/html/site1/  
index.html  style_index.css
```

N.B. : Le terminal est placé actuellement dans `/etc/apache2/sites-available`, mais cela ne nous empêche pas de voir le contenu de `site1` car nous utilisons un chemin absolu : la commande commence par un « / » (racine de l'ordinateur). Il est intéressant à remarquer que pour les copies « **cp** » nous avons eu recours deux fois à cette méthode dans chaque commande.

Dernière étape : vérifier que le site a bien été **configuré**. Saisissez les commandes ci-dessous pour éviter tout problème :

```
bastien@bastien:~$ sudo chown -R www-data:www-data /var/www/html/site1
```

```
bastien@bastien:~$ sudo apache2ctl configtest  
Syntax OK
```

« **configtest** » va vérifier s'il y a aucune erreur de syntaxe. Par exemple, une erreur de syntaxe dans un fichier de configuration donnera :

```
bastien@bastien:/etc/apache2/sites-available$ sudo apache2ctl configtest  
AH00112: Warning: DocumentRoot [/var/www/html/site1] does not exist  
Syntax OK
```

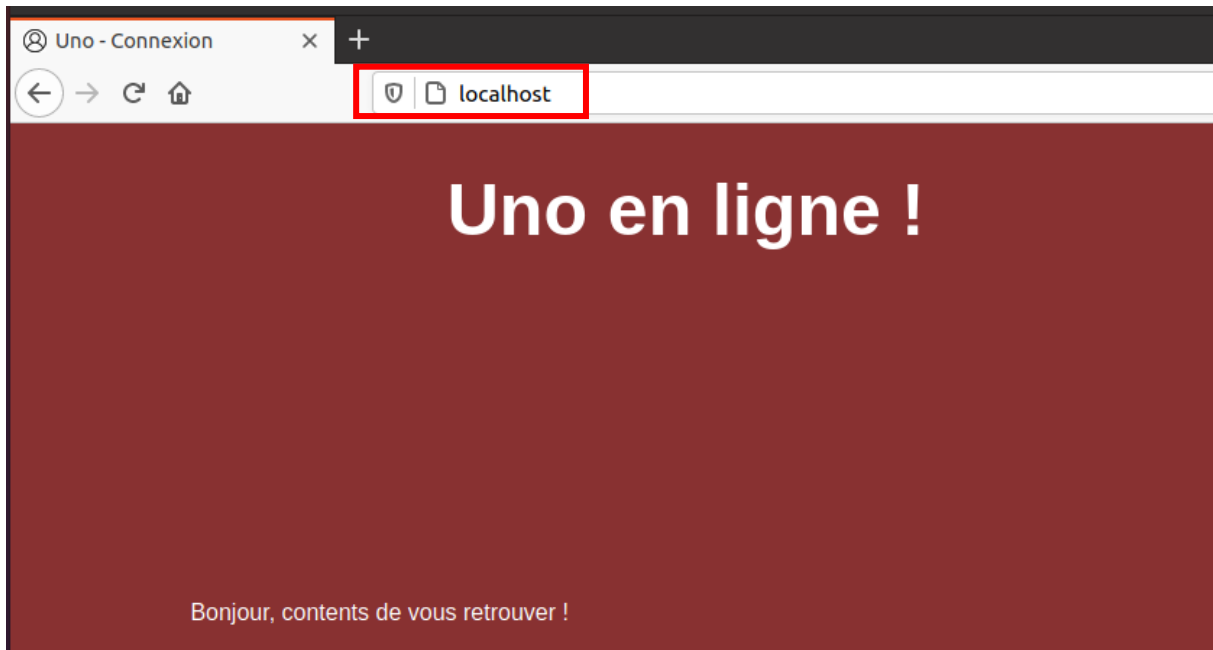
Enfin, **redémarrez** le serveur apache pour qu'il prenne en considération les modifications :

```
bastien@bastien:/etc/apache2/sites-available$ sudo systemctl reload apache2
```

1.3.3 Lancement du site internet en local

Maintenant que les fichiers sont bien présents dans le dossier spécifié dans le DocumentRoot, il suffit simplement d'ouvrir le **navigateur** (n'importe lequel). Par défaut, Ubuntu propose un site local en indiquant « **localhost** » dans l'URL. Tapez donc « localhost » dans votre URL. Si tout est correct, votre site devrait apparaître, avec potentiellement son style.

Exemple :



Dans notre exemple, localhost affiche bien le site copié, ainsi que son css, dans le répertoire donné au DocumentRoot. Le site est donc mis en **local**.

1.4 Lancement de WireShark

1.4.1 Définition des trames

Les **trames internet** sont des **communications** entre votre ordinateur et le serveur. C'est comme ceci que vous pouvez avoir un affichage d'un site. Si vous êtes sur un site et que vous demandez d'aller dans un second site, grâce à une **identification** (identifiant + mot de passe), alors votre ordinateur va envoyer ses informations et demander au serveur d'accéder à ce site. C'est aussi pour cela que le protocole **https** est **sécurisé** : dans les informations d'identifications que l'ordinateur donne, il y a aussi l'identifiant et le mot de passe **transmis**. Avec http, ces informations sont visibles en **brut**, alors qu'avec https, elles sont **cryptées**.

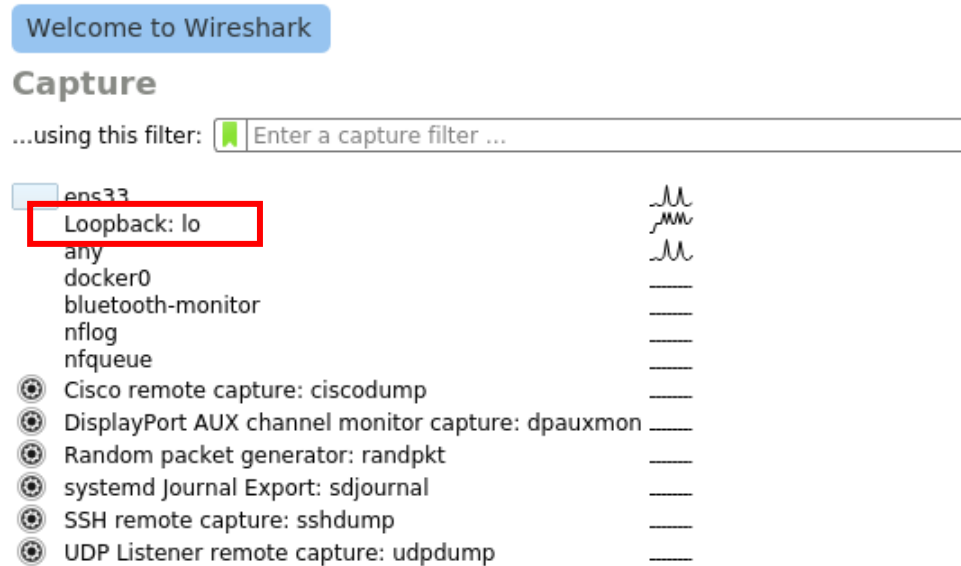
1.4.2 Accéder aux trames internet

Pour ce faire, il vous faut avoir Wireshark, disponible de la même méthode que apache2, dans l'apt. Pour l'installer, faites dans le terminal :

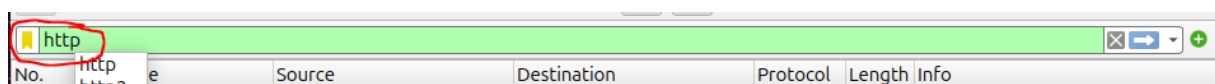
```
sudo apt install WireShark
```

Après avoir fait [Entrée], **Wireshark** est installé et vous pouvez y accéder dans votre liste d'applications ou grâce à la commande (une méthode n'impacte en rien les performances) :

```
devops@ubuntu:/etc/apache2/sites-available$ sudo wireshark
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runtime-root'
```



Voici l'interface de l'application. Il faut sélectionner ici Loopback : lo car localhost est une adresse IP en 127.X.Y.Z où le trafic revient vers nous. Entrez le **filtre http** pour ne voir que les trames http (celles qui nous intéressent)



Ensuite, **rechargez** la page. Seront ensuite présentées toutes les trames entre le site et votre ordinateur.

No.	Time	Source	Destination	Protocol	Length	Info
56	17.106958016	127.0.0.1	127.0.0.1	HTTP	550	GET / HTTP/1.1
58	17.107683830	127.0.0.1	127.0.0.1	HTTP	8783	HTTP/1.1 200 OK (text/html)
60	17.307045453	127.0.0.1	127.0.0.1	HTTP	498	GET /style_index.css HTTP/1.1
61	17.307891897	127.0.0.1	127.0.0.1	HTTP	1160	HTTP/1.1 200 OK (text/css)

4 trames sont identifiées et respectivement : **la trame de demande de la page html**, avec sa **réponse** ainsi que la demande du css (et réponse). Si jamais vous ne voyez pas les **2 dernières trames**, c'est que le cache de votre navigateur est activé et que la page a déjà été chargée.

Sur la trame numéro 2, la trame de réponse, peut être aperçu le type de donnée transmis par le serveur. Dans l'illustration, c'est du text/html. A côté, peut être vu 200 OK qui est un code de retour : **entre 200 et 206 le code est positif** contrairement au code **403**, par exemple, qui signifie que des **droits** sont nécessaires (le code **404** signifie que la ressource **n'existe pas**).

1.5 Passage au port 8080

La seule différence entre les **ports 80 et 8080** est l'Host qui va changer. En effet, si un utilisateur souhaite accéder à notre site web sur le port 8080, il ne faut plus seulement spécifier **localhost** mais **localhost:8080** pour indiquer au navigateur que l'utilisateur veut lancer le site sur le port 8080. Il n'y a donc aucune différence ? Si, seulement elle n'est pas visible. Premièrement, un site web sur le port 8080 est souvent un site web en **cours de développement**. Cela est communément appelé un « **port de version de test du site** ». Deuxièmement, quand un serveur web est sur le port 80 il n'y a pas besoin de spécifier le port contrairement au port 8080.

Pour insérer le port 8080, nous allons rajouter un « VirtualHost » pour le site configuré. Pour cela, il faut se rendre dans le répertoire : `/etc/apache2/sites-available/` et modifier le fichier **000-default.conf** avec les commandes suivantes :

```
bastien@bastien:/$ cd /etc/apache2/sites-available/
bastien@bastien:/etc/apache2/sites-available$ sudo nano 000-default.conf
```

Une fois dans le fichier, il faut rajouter ces lignes. Cela va spécifier une autre VirtualHost mais cette fois-ci sur le port 8080. Pour le DocumentRoot voir l'installation des sites (cf. 1.3.2).

Cependant il faut aussi que le « **Listener** » écoute sur le port 8080 en plus du port 80. Pour cela, il faut aller dans le fichier « ports.conf » qui se situe dans le répertoire : /etc/apache2 voici les commandes pour y accéder et le modifier :

```
<VirtualHost *:8080>

    ServerName localhost
    DocumentRoot /var/www/html/site1

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

</VirtualHost>

bastien@bastien:/etc/apache2/sites-available$ cd ../
bastien@bastien:/etc/apache2$ ls
apache2.conf  conf-enabled  magic          mods-enabled  sites-available
conf-available  envvars      mods-available  ports.conf    sites-enabled
bastien@bastien:/etc/apache2$ sudo nano ports.conf
```

Une fois dans le fichier, faites « **nano** » et écrivez cette ligne pour que le serveur écoute sur le port 80 et 8080. « **Ecouter** » signifie pouvoir accéder, en quelques sortes.

```
Listen 80
Listen 8080
```

Dans WireShark :

```
Transmission Control Protocol
Hypertext Transfer Protocol
  GET / HTTP/1.1\r\n
    Host: localhost\r\n
    User-Agent: Mozilla/5
    Accept: text/html,application/xhtml+xml,application/javascript;q=0.9,*/*;q=0.8\r\n
```

Port 80

```
Transmission Control Protocol, Src Port: 39138
Hypertext Transfer Protocol
  GET / HTTP/1.1\r\n
    Host: localhost:8080\r\n
    User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:52.0) Gecko/20100101 Firefox/52.0\r\n
    Accept: text/html,application/xhtml+xml,application/javascript;q=0.9,*/*;q=0.8\r\n
    Accept-Language: en-US,en;q=0.5\r\n
    Accept-Encoding: gzip, deflate, br, zstd\r\n
```

Port 8080

1.6 Capture de formulaire GET/POST

Pour cette partie, il faut obligatoirement un site internet ayant un formulaire.

Sur la page déployée, il y a un formulaire. Voyons s'il y a une différence entre la méthode **GET** et **POST**.

Quand des informations sont fournies dans le formulaire avec la méthode **GET** et qu'elles sont soumises, la page va être rechargée avec les **attributs** (ici, sur l'image name et password). Le CSS est rechargé car le **cache** du navigateur a été **désactivé**.

No.	Time	Source	Destination	Protocol	Length	Info
4	0.000091644	127.0.0.1	127.0.0.1	HTTP	638	GET /?name=bastien&password=admin HTTP/1.1
6	0.001201194	127.0.0.1	127.0.0.1	HTTP	8150	HTTP/1.1 200 OK (text/html)
8	0.279711674	127.0.0.1	127.0.0.1	HTTP	526	GET /style_index.css HTTP/1.1
9	0.280296615	127.0.0.1	127.0.0.1	HTTP	1160	HTTP/1.1 200 OK (text/css)

Alors qu'avec la méthode **POST**, les attributs du formulaire sont passés pas dans l'**URL**, mais directement dans le corps de la **requête**, comme visible sur l'image suivante :

No.	Time	Source	Destination	Protocol	Length	Info
7	14.603612440	127.0.0.1	127.0.0.1	HTTP	708	POST / HTTP/1.1 (application/x-www-form-urlencoded)
9	14.605341995	127.0.0.1	127.0.0.1	HTTP	8807	HTTP/1.1 200 OK (text/html)
11	14.884157590	127.0.0.1	127.0.0.1	HTTP	498	GET /style_index.css HTTP/1.1
12	14.884622274	127.0.0.1	127.0.0.1	HTTP	1160	HTTP/1.1 200 OK (text/css)

Frame 7: 708 bytes on wire (5664 bits), 708 bytes captured (5664 bits) on interface 0	0000	00 00 00 00 00 00 00 00	00 00 00 00 08 00 45 00	...
Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00	0010	02 b6 60 34 40 00 00 06	da 0b 7f 00 00 01 7f 00	...
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1	0020	00 01 8f 20 00 50 5c c3	e3 fe 05 03 c1 f4 80 18	...
Transmission Control Protocol, Src Port: 36640, Dst Port: 80, Seq: 1,	0030	02 00 00 ab 00 00 01 01	08 0a 3d f7 35 d1 3d f7	...
Hypertext Transfer Protocol	0040	35 d1 50 4f 53 54 20 2f	20 48 54 54 50 2f 31 2e	5. PO
Host: localhost	0050	31 0d 0a 48 6f 73 74 3a	20 6c 6f 63 61 6c 68 6f	1..t
Content-Type: application/x-www-form-urlencoded	0060	73 74 0d 0a 55 73 65 72	2d 41 67 65 6e 74 3a 20	st..
Form item: "name" = "bastien"	0070	4d 6f 7a 69 6c 6c 61 2f	35 2e 30 20 28 58 31 31	Moz:
Form item: "password" = "admin"	0080	2b 20 55 62 75 6a 74 75	2b 20 4c 60 6a 75 78 20	.. 10

1.7 Configuration avec Telnet

Telnet (**TE**letype **NE**twork) est un protocole réseau permettant de se connecter à distance à un serveur via une connexion en ligne de commande.

Pour intégrer le protocole Telnet, il faut l'installer grâce à la commande :

```
bastien@bastien:/etc/apache2$ sudo apt install telnet
```

Ensuite, il faut lancer la connexion telnet avec la commande :

```
telnet <site> <port>
```

Dans notre cas, le site est localhost et le port 80

```
bastien@bastien:/etc/apache2$ telnet localhost 80
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^['.

```

Enfin, il faut taper ces 2 lignes et appuyer 2 fois sur [Entrée].

Sera ainsi reçu le html du site associé. De la même manière, le css associé peut être récupérer :

```
bastien@bastien:/etc/apache2$ telnet localhost 80
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
GET / HTTP/1.1
Host:localhost
```

```
bastien@bastien:/etc/apache2$ telnet localhost 80
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
GET /style_index.css HTTP/1.1
Host:localhost
```

2. Configuration d'un second site avec un DNS et des Hosts

Pour configurer un second site, il faut créer des **sous-domaines**. Un sous-domaine est une **extension** d'un domaine principal, permettant de séparer différentes sections d'un site web. Ici, nous avons configuré un site sur le domaine localhost. Pour le deuxième site, nous pouvons, par exemple, le déployer sur le domaine **site2.local** ou **site2.localhost**. Pour cela plusieurs, il y a plusieurs étapes :

- 1) Créer un fichier de configuration :

```
bastien@bastien:/$ sudo touch /etc/apache2/sites-available/site2.conf
```

- 2) Ecrire dedans :

```
bastien@bastien:/etc/apache2/sites-available$ sudo nano site2.conf
```

Ici, **ServerName** est une **directive** dans la configuration d'Apache qui définit le nom principal ou l'adresse du serveur web. Il indique à Apache quel nom de domaine ou sous-domaine il doit écouter et associer à un certain VirtualHost.

```
<VirtualHost *:80>

    ServerName site2.local
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/site2
    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

</VirtualHost>
```

Comme dit au-dessus, le site va s'appeler site2.local, cela signifie que l'utilisateur va pouvoir accéder au site web en tapant site2.local.

- 3) Ensuite, il faut activer le site avec **a2ensite Nom_du_site**. La commande permet **d'activer** un site en créant un **lien symbolique** entre un fichier de configuration dans sites-available et le répertoire site-enabled. Cela permet à Apache de prendre en compte et de servir ce site.

```
bastien@bastien:/etc/apache2/sites-available$ sudo a2ensite site2.conf
Enabling site site2.
To activate the new configuration, you need to run:
    systemctl reload apache2
```

- 4) Aller dans /etc/hosts et modifier le en rajoutant le ligne ci-dessous.

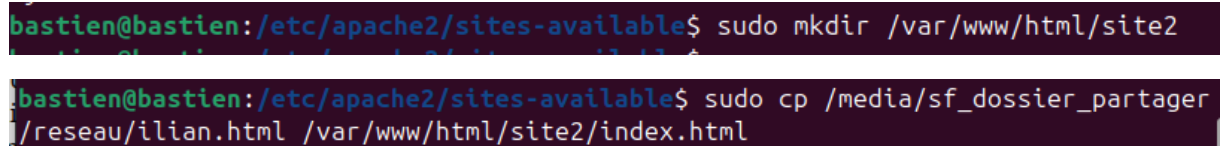
```
bastien@bastien:/etc$ sudo nano hosts
```

Cette ligne permet au navigateur de pouvoir résoudre le nom site2.local en utilisant l'adresse **127.0.0.1**. **site2.local** devient donc un alias a **127.0.0.1**.



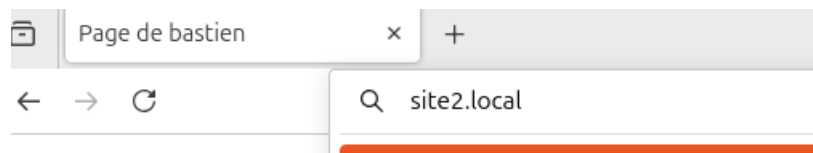
```
GNU nano 7.2 hosts *
127.0.0.1 localhost
127.0.1.1 bastien
127.0.0.1 site2.local
```

- 5) Il ne reste plus qu'à ajouter un site web. Il va falloir créer le dossier site2 a /var/www/html/ et mettre notre fichier html dedans.



```
bastien@bastien:/etc/apache2/sites-available$ sudo mkdir /var/www/html/site2
bastien@bastien:/etc/apache2/sites-available$ sudo cp /media/sf_dossier_partager/reseau/ilian.html /var/www/html/site2/index.html
```

- 6) Entrez site2.local dans votre navigateur et le site web s'affiche :

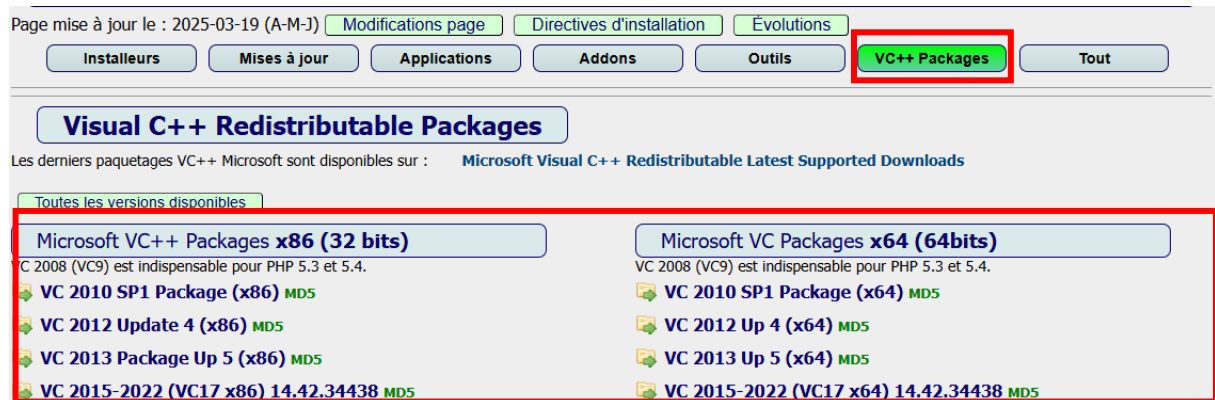


3. Configuration d'un serveur local Windows (WAMP)

3.1 Installation de WampServer

Pour installer **WampServer**, il faut avoir installé **VC++** (dépendance de WampServer, nécessaires pour pouvoir démarrer un serveur).

Installation : Sur <https://wampserver.aviatechno.net/> dans VC++ package, il faut installer les 8 packages.



3.2 Dépôt des fichiers dans le serveur local

Après avoir tout installé, ouvrez votre terminal Windows. Pour cela, appuyez sur votre touche **[Windows]** ou en bas à droite de votre barre de tâche. Tapez « **cmd** » et faites entrer. Pour, Windows, les commandes de déplacement restent « **cd** », mais pour afficher le contenu d'un dossier, c'est « **dir** » :

```
C:\Users\etulyon1>cd \
C:\>dir
Le volume dans le lecteur C s'appelle Windows
Le numéro de série du volume est 54E9-D70D

Répertoire de C:\

31/12/2024  10:50    <DIR>          back_up
27/12/2024  22:18    <DIR>          doxygen-1.12.0.src.tar
27/12/2024  22:14             8 472 577 doxygen-1.12.0.src.tar.gz
17/03/2025  07:02    <DIR>          Intel
07/12/2019  10:14    <DIR>          PerfLogs
08/03/2025  14:52    <DIR>          Program Files
06/02/2025  23:42    <DIR>          Program Files (x86)
13/12/2024  09:03    <DIR>          Riot Games
16/01/2025  13:35             587 604 133 sqldeveloper-24.3.1.347.1826-no-jre.zip
22/11/2024  15:38    <DIR>          Users
16/03/2025  16:23    <DIR>          wamp64
16/03/2025  23:08    <DIR>          Windows
25/11/2024  10:53    <DIR>          xampp

                2 fichier(s)          596 076 710 octets
                11 Rép(s)  294 334 640 128 octets libres

C:\>cd wamp64
```


Allez dans le répertoire de WampServer, et dans son sous-répertoire pour stocker les pages web avec cette commande : « `cd \wamp64\www` ». Ensuite, créez un répertoire nommé par le nom de votre choix. Dans l'exemple suivant, le nom choisi est site1. Il faut effectuer la commande : « **mkdir [nom_repertoire]** ».

```
C:\wamp64\www>mkdir site1

C:\wamp64\www>dir
Le volume dans le lecteur C s'appelle Windows
Le numéro de série du volume est 54E9-D70D

Répertoire de C:\wamp64\www

19/03/2025  10:09    <DIR>          .
19/03/2025  10:09    <DIR>          ..
02/01/2024  10:06           49 189 add_vhost.php
31/12/2010  08:40        202 575 favicon.ico
19/08/2024  15:10           30 252 index.php
19/03/2025  10:09    <DIR>          site1
17/06/2021  15:48          810 testmysql.php
21/09/2015  17:30          742 test_sockets.php
16/03/2025  16:20    <DIR>          wamplanguages
16/03/2025  16:20    <DIR>          wampthemes
               5 fichier(s)          283 568 octets
               5 Rép(s)  294 335 111 168 octets libres
```

Ensuite, il faut copier les sites à déposer dans site1. Pour ce faire, il faut connaître le répertoire source et le répertoire destination. Il est conseillé de rester dans « `\wamp64\www` » pour que le répertoire destination soit toujours de la forme « `.[nom_dossier]` » (où `nom_dossier` représente le dossier créé ci-dessus). Dans l'exemple, le fichier source est dans un périphérique de stockage externe d'où « `E:\` ». S'il se situait sur le disque dur de l'ordinateur, le « `E:\` » serait un « `C:\` ».

En guise d'exemple, le fichier déplace se situe dans un répertoire nommé « `jeu` » dans un périphérique de stockage externe. Le fichier s'intitule `index.html` :

```
C:\wamp64\www>copy E:\jeu\index.html .\site1\
1 fichier(s) copié(s).
```

S'il y a un css de nécessaire, il faut faire la même commande en changeant le nom du fichier :

```
C:\wamp64\www>copy E:\jeu\style_index.css .\site1\
1 fichier(s) copié(s).
```

Pour la suite, il faut voir la **version d'apache** de votre ordinateur. Premièrement, allez dans wamp64 avec la commande « `cd C:\wamp64` ». Deuxièmement, entrez la commande suivante :

```
C:\wamp64>dir \wamp64\bin\apache\
Le volume dans le lecteur C s'appelle Windows
Le numéro de série du volume est 54E9-D70D

Répertoire de C:\wamp64\bin\apache

16/03/2025  16:21    <DIR>          .
16/03/2025  16:21    <DIR>          ..
16/03/2025  16:21    <DIR>          apache2.4.62.1
16/03/2025  16:21    <DIR>          modules_sup
                0 fichier(s)                0 octets
                4 Rép(s)  294 326 681 600 octets libres
```

Dans l'exemple ci-dessus, la version d'apache est : « **apache2.4.62.1** ».

Grâce à cela, le terminal indique que le fichier existe bien. Avec le relevé de la version d'apache, il faut aller dans le répertoire avec sa version de spécifié, comme suit :

```
C:\wamp64\bin\apache>cd C:\wamp64\bin\apache\apache2.4.62.1\conf\extra\
```

Version d'apache actuellement sur le PC

Dès lors que vous êtes situés dans le répertoire, il faut l'éditer. L'éditeur de texte sera **notepad**, effectuez donc cette commande :

```
C:\wamp64\bin\apache\apache2.4.62.1\conf\extra>notepad httpd-vhosts.conf
```

Une fois dans le notepad, il faut ajouter précisément ces lignes, à l'exception de « site1 » qui doit être **remplacé** par le répertoire créé précédemment. Le ServerName représente le nom du site.

```
<VirtualHost *:80>
    ServerName site1.local
    DocumentRoot "C:/wamp64/www/site1"
    <Directory "C:/wamp64/www/site1">
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

Quand cela est fait, il faut enregistrer et fermer la fenêtre.

Pour la suite, il faut que l'ordinateur puisse résoudre le nom du site. Tapez cette commande :

```
C:\>notepad C:\Windows\System32\drivers\etc\hosts
```

Une fois dans le notepad, il suffit de rajouter cette ligne. Dans l'exemple, site1 est gardé. Si un autre nom de site du ServerName est spécifié, l'écrire sous la forme : [nom_site].local.

```
127.0.0.1 site1.local|
```

3.3 Lancement et test du serveur Windows

Pour **arrêter** le serveur, tapez cette commande :

```
C:\>net stop wampapache64
```

Pour le **refaire tourner**, faire :

```
C:\>net start wampapache64
```

N.B. : Contrairement à GNU/Linux, il n'existe pas de **reload** pour les modifications. Il faut donc passer par ces deux commandes ci-dessus.

Enfin, le site est prêt. Tapez le nom du site que vous venez de mettre dans votre navigateur :



Le site devrait s'afficher normalement :



4. Mise en place d'un serveur avec Nginx

4.1 Installation de Nginx

Pour configurer **NGINX** il faut déjà l'installer si cela n'est pas déjà fait. Il faut effectuer la commande ci-dessous :

```
bastien@bastien:~$ sudo apt install nginx
[sudo] password for bastien:
```

Faites [Entrée]. Comme pour apache2, des lignes de commandes s'exécutent, mais il suffira seulement de faire Y et [Entrée].

4.2 Configuration de sites pour Nginx

De la même manière que pour apache, il faut déposer votre site sur `/var/www/html/site1`. Ici pour, cet exemple, le site utilisé est comme pour celui d'apache, et il s'appellera « site1 ». Le choix du nom du fichier reste **toujours à la disposition de l'utilisateur**. Il faut d'abord créer un fichier de configuration avec la commande ci-dessous :

```
sudo nano /etc/nginx/sites-available/site1
```

Dans ce fichier, saisir les lignes suivantes en veillant à respecter le **serveur_name** (alias `ServerName` pour apache2). Ici, le principe de **DNS** ne sera pas appliqué et le DNS par défaut sera le **DNS localhost**.

```
server {
    listen 80;
    server_name site1.localhost;

    root /var/www/html/site1;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Ensuite, contrairement à apache2 qui le fait tout seul quand la commande « **a2ensite MonSite** » est saisie, avec NGINX il faut créer ce lien manuellement avec la commande « **ln** » et l'option « **-s** » (l'option « **-s** » de la commande `ln` sous Linux sert à créer un lien symbolique). Le premier chemin est **votre site** et le deuxième chemin là où **le lien va être créé**.

```
bastien@bastien:~$ sudo ln -s /etc/nginx/sites-available/site1 /etc/nginx/sites-enabled/
```

Après avoir tout configuré, il faut vérifier **qu'aucune erreur de syntaxe ou autre erreur** (telle que l'oubli de l'une des étapes précédente). Pour cela, exécutez la commande ci-dessous :

```
bastien@bastien:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

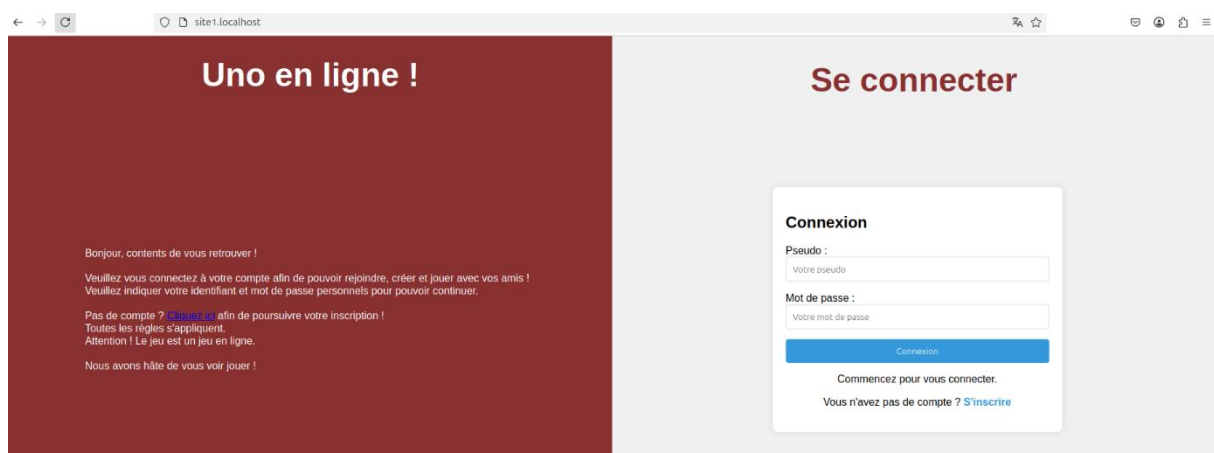
N.B. : Si une erreur n'a pas été traitée dans cette notice, veuillez-vous référer à cette **notice**, plus complète sur le sujet : [Notice NGINX](#)

4.3 Lancement et vérification du site en local

Étant donné qu'un seul serveur peut tourner, vérifiez que **apache2** n'est pas actif sur votre machine. Pour être sûr du bon lancement de NGINX, exécutez les 2 commandes ci-dessous pour **arrêter** apache2 (s'il est actif) et **lancer NGINX**.

```
bastien@bastien:~$ sudo systemctl stop apache2
bastien@bastien:~$ sudo systemctl start nginx
```

Enfin, saisissez votre site sur le navigateur avec **site1.localhost** (pour notre exemple) ou simplement **localhost** (selon les configurations du site et du DNS).



Une dernière vérification est nécessaire pour s'assurer que c'est bien NGINX qui loue le serveur et pas apache2. Pour cela, relancez Wireshark et mettez-le en **écoute** (cf. 1.4) et lancez votre site. Une fois dans Wireshark, cliquez sur la requête de réponse à la demande de la page WEB. Comme ceci :

```
6 0.001122081 127.0.0.1 127.0.0.1 HTTP 256 HTTP/1.1 304 Not Modified
```

Une fois cliqué, du contenu s'affiche en bas de votre fenêtre. Il faut dérouler **Hypertext Transfer Protocol** et vous devrez voir Server : **NGINX**. Si ce n'est pas le cas remonter 2 étapes au-dessus.

```

> Frame 6: 256 bytes on wire (2048 bits), 256 bytes captured (2048 bits) on interface lo, 10 0
> Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 80, Dst Port: 53906, Seq: 1, Ack: 532, Len: 190
- Hypertext Transfer Protocol
  > HTTP/1.1 304 Not Modified\r\n
    Server: nginx/1.24.0 (Ubuntu)\r\n
    Date: Wed, 19 Mar 2025 10:11:15 GMT\r\n
    Last-Modified: Sun, 16 Mar 2025 12:56:21 GMT\r\n
    Connection: keep-alive\r\n
    ETag: "67d6ca75-2e63"\r\n
    \r\n
    [HTTP response 1/1]
    [Time since request: 0.000119858 seconds]
    [Request in frame: 4]
    [Request URI: http://site1.localhost/]

```

5. Mise en place d'un site sécurisé avec SSL

Vous pouvez constater ci-dessous que le site s'ouvre en **http** (sans le petit cadenas). Par conséquent, les échanges peuvent être interceptés et lus étant donné qu'ils ne sont pas chiffrés. Ici, cela ne pose pas de problème puisque que le site est en local. Cependant, si vous voulez déployer votre site en **ligne**, il faut **sécuriser** son accès. Voici comment faire en version simplifiée.



Pour cela il faut rajouter le « **VirtualHost** » du site sur le **port 443**, port pour https. Voici comment y procéder :

Premièrement, il suffit d'ajouter le VirtualHost du fichier site2.conf comme ceci :

```
<VirtualHost *:443>

    ServerName site2.local
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/site2

    #ligne qui change par rapport au port 80
    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/site2.crt
    SSLCertificateKeyFile /etc/ssl/private/site2.key
    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

</VirtualHost>
```

Pour que le site soit sécurisé, il faut un **certificat** ainsi qu'une **clé de chiffrement pour SSL**. Vérifiez d'abord que notre votre **écoute** bien sur le port 443.

Deuxièmement, pour vérifier que le serveur écoute bien sur le port 443, Il faut aller dans le fichier `/etc/apache2/ports.conf`. Une fois dans le fichier, assurez-vous que les lignes suivantes sont présentes sinon rajoutez-les :

```
<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>
```

Troisièmement, le module SSL, qui est le module de connexion sécurisée, doit être activée. De plus, une **clé privée configurée** ainsi qu'un **certificat SSL** pour chiffrer les échanges entre le serveur et le client, garantissant ainsi la confidentialité et l'intégrité des données, doivent être mis en place. Dans les commandes suivantes, le répertoire où vous vous trouvez n'a pas d'importance.

- 1) Activez le **module SSL** avec la commande suivante :

```
bastien@bastien:/etc/apache2/sites-available$ sudo a2enmod ssl
```

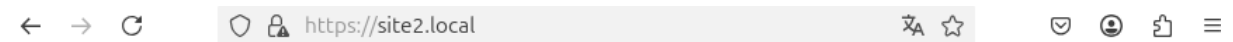
- 2) Créez une **clé privée** et le **certificat** qui va avec :

```
bastien@bastien:/etc/apache2$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/site2.key -out /etc/ssl/certs/site2.crt
```

- 3) Vérifiez que le **certificat** ait bien été créé. Pour cela, tapez la commande suivante (si une ligne s'affiche comme ci-dessous, alors tout est dans les normes) :

```
bastien@bastien:/etc/apache2$ ls -l /etc/ssl/certs/site2.crt
-rw-r--r-- 1 root root 1407 mars 14 16:10 /etc/ssl/certs/site2.crt
```

- 4) Il vous reste à vérifier que la configuration est bonne avec les commandes (**cf 1.3.2** dernière étape) en veillant à remplacer site1 par site2 dans les commandes.
- 5) Maintenant, le site web est accessible de façon **sécurisée**. En saisissant `https://site2.local` dans notre navigateur.



Page de l'auteur : Ilian boussaid

Fonction remplacement

Ici est présenté une partie de la fonction permettant de placer les joueur sur la grille.

```
def remplacementJ4():
    global Joueur4, J4, grille_jeu, place_J1, place_J2, place_J3, place_J4, nombre_J4, lancer_deJ4, placement4
    if placement4 == 1:
        position_grille_jeu_tour_suivant(4)
```

Annexe

<code>sudo apt update</code>	Met à jour la liste des paquets disponibles
<code>sudo apt install <package></code>	Installe un paquet
<code>sudo systemctl start <service></code>	Démarre un service
<code>sudo systemctl stop <service></code>	Arrête un service
<code>sudo systemctl restart <service></code>	Redémarre un service
<code>sudo systemctl reload <service></code>	Recharge la configuration sans redémarrer
<code>sudo systemctl status <service></code>	Vérifie l'état d'un service
<code>sudo apachectl configtest</code>	Vérifie la configuration d'Apache
<code>sudo nginx -t</code>	Vérifie la configuration de Nginx
<code>cd <dossier></code>	Change de répertoire
<code>ls <dossier> <fichier></code>	Liste les fichiers d'un répertoire
<code>nano <fichier></code>	Ouvre un fichier dans l'éditeur Nano
<code>mkdir <dossier></code>	Crée un répertoire
<code>cp <source> <destination></code>	Copie un fichier ou un dossier
<code>sudo a2ensite <site.conf></code>	Active un VirtualHost Apache
<code>sudo a2dissite <site.conf></code>	Désactive un VirtualHost Apache
<code>nano /etc/hosts</code>	Modifie le fichier hosts pour associer un nom de domaine local
<code>sudo ln -s <source> <lien_symbolique></code>	Crée un lien symbolique
<code>sudo a2enmod ssl</code>	Active le module SSL d'Apache
<code>sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout <clé> -out <certificat></code>	
Génère un certificat SSL auto-signé :	
- <code>`-x509`</code> : Génère un certificat auto-signé	

- ``-nodes`` : Pas de chiffrement de la clé privée
- ``-days 365`` : Valable 1 an
- ``-newkey rsa:2048`` : Crée une nouvelle clé RSA 2048 bits

`telnet <site> <port>`

Ouvre une connexion Telnet vers un site sur un port donné

`sudo wireshark`

Ouvre Wireshark avec les droits administrateur